



# Reconstruction de Courbes de Charge Challenge Enedis

**Intelligence Artificielle**

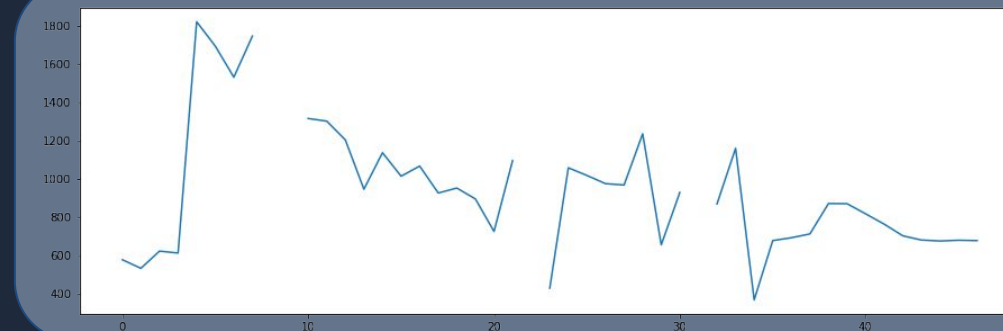
2025-2026

### Le Défi Enedis

- **38 Millions** de compteurs Linky.
- Problème : Pertes de données techniques (transmission, stockage) créant des **"trous"**.
- **Mission** : Reconstruire les données manquantes de manière réaliste.

Données : 21 000 courbes (DeepCoubogen)  
Métrique : MAE (Mean Absolute Error)

Courbe 'Holed\_1' sur les 24 premières heures



### Benchmark : Interpolation Linéaire

Méthode géométrique simple fournie par défaut.

#### ⚠ Limites

Trace une ligne droite entre A et B.

**Incapable** de capturer les pics de consommation  
(allumage d'appareils, chauffage).

Objectif : Comprendre la "logique" électrique, pas  
juste la géométrie.



Réalité (pointillés) vs Linéaire  
(rouge)

### Architecture TensorFlow (MLP)

- Approche "Fenêtre glissante" (Windowing).
- Entrée : 6 points avant + 6 points après.
- Couches Dense (128, 64, 32 neurones).

### Problème

Le réseau de neurones est moins performant pour prédire des grandes plages de données manquantes

#### Problème de prédiction sur le trou horaire :

Plus le trou est grand, plus il va regarder des point avant et après le trou qui n'ont rien à voir.

```
# Création des fenêtres glissantes
for t in range(window_size, len(curve) - window_size):
    # Contexte Avant + Après
    before = curve[t-window_size : t]
    after = curve[t+1 : t+1+window_size]

    features = np.concatenate([before, after])
    target = curve[t]

    X_list.append(features)
    y_list.append(target)
```

### Méthode de base

- **Modèle Unique**

Prend ses propres décisions en divisant les données étape par étape.

- **Par régression**

L'objectif est de prédire une valeur continue (un nombre) prenant n'importe quelle valeur dans une plage

- **Problème d'overfitting**

Modèle très sensible aux variations des données  
⇒ surapprentissage (Overfitting)

**126.32**

MAE (WATTS)

Score Hackathon

### Méthode d'Ensemble

- **n Arbres de Décision**

Modifiable avec le paramètre `n_estimators` de la fonction `RandomForestRegressor`

- **Entraînement**

Chaque arbre s'entraîne sur un sous-ensemble aléatoire de données et prend la moyenne de tous les arbres.

- **Avantages**

Moins d'overtfitting, plus stable, meilleure corrélations entre les colonnes...

- **Inconvénients**

Peut vite devenir long à l'exécution (lourd en calcul)



```
X_explicatives = X_test_clean.iloc[:, :n]
colonnes_trouvees = X_test_clean.columns[-1000:]

...

model = RandomForestRegressor(n_estimators=25, max_depth=15,
                             random_state=42, n_jobs=-1)

model.fit(X_train_courante, y_train_courante)

valeurs_predites = model.predict(X_predict_courante)
```

### Performance Optimale

Le modèle capture les **effets de paliers** (sauts de consommation) là où le Deep Learning lissait trop.

Voici les résultats sur 50 courbes aléatoires :

**89.7**

MAE (WATTS)

Division par 2 de l'erreur

Prédiction RF



Capacité à reproduire les sauts brutaux

## Ce qu'il faut retenir

- **Complexité  $\neq$  Performance** : Le RF (simple) bat le DL (complexe) sur ce format tabulaire.
- **Robustesse** : La méthode d'ensemble annule le bruit.

## Pour aller plus loin...

XGBoost

Gradient Boosting pour corriger les erreurs séquentielles.

Saisonnalité

Élargir la fenêtre à 24h (cycle journalier).